Guide RAG / BrainVault pour débutant — comprendre et surveiller la partie IA de Riemann_Lab

Fichier : Guide-RAG-BrainVault-Debutant.md · **Dossier :** wiki (racine) **Branche :** master (wiki) · **Auteur :** hprzeta · **MAJ :** 2026-11-11

Rôle de cette page. Expliquer **sans jargon** comment fonctionne le cerveau IA du projet (le RAG « BrainVault » sur le SSD `/mnt/vault_rag`), ce que sont tous ces fichiers mystérieux (`chroma.sqlite3`, `data_level0.bin`, `docstore.json…`), et comment **surveiller** le système quand il tourne avec le script `rag_monitor.py`. Tous les exemples sont **réels** : ce sont les tests effectués les 4-5 juillet 2026 sur PC1.

---

1. C'est quoi un RAG ? (l'image du bibliothécaire)

RAG = *Retrieval-Augmented Generation* — « génération augmentée par récupération ».

Imagine un bibliothécaire (le modèle IA, ex. **mathstral**) très intelligent mais qui n'a **jamais lu tes documents**. Si tu lui demandes « quel est le seuil SEUIL_1NEWTON ? », il ne peut pas le savoir : cette valeur n'existe que dans TES fichiers (wiki, analyses, code).

Le RAG résout ça en 2 temps :

1. **Retrieval (récupération)** : avant de répondre, le système **cherche dans ta bibliothèque** les 3-5 passages les plus pertinents pour la question.
2. **Generation (génération)** : il donne ces passages au modèle IA avec la consigne « réponds à la question EN T'APPUYANT sur ces extraits ».

Résultat : le modèle répond avec **tes** données, à jour, et peut **citer ses sources**. Sans RAG, il inventerait (« hallucination »). Avec RAG, il consulte.

> C'est exactement pour ça qu'on a corrigé les IP `.94->.52` et posé les bandeaux ARCHIVE avant l'ingestion : **le RAG croit ce qu'on lui donne**. Documents faux = réponses fausses.

---

2. Le vocabulaire IA — traduit en français simple

Terme	Traduction simple	Dans Riemann_Lab
LLM	Le « cerveau » : un modèle de langage qui lit et écrit du texte	`mathstral` (spécialisé maths), `deepseek-coder`, `qwen3` via Ollama

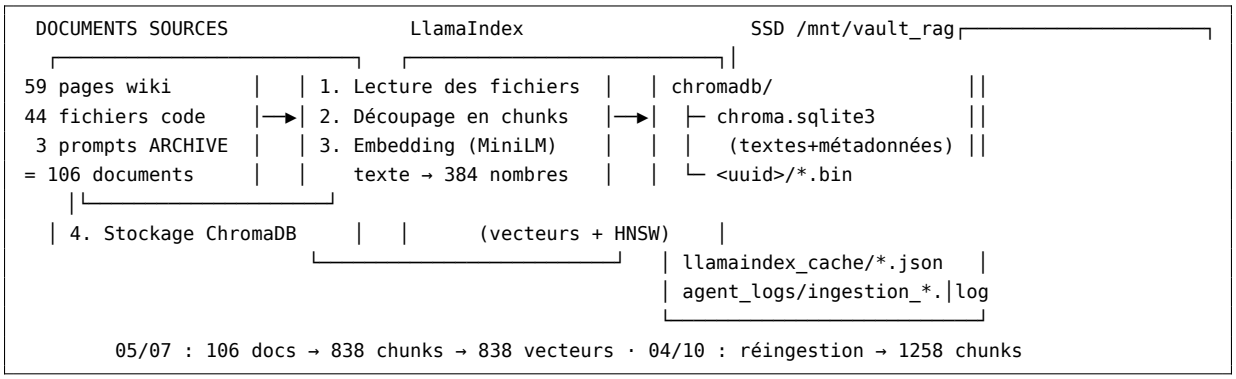
Terme	Traduction simple	Dans Riemann_Lab
Ollama	Le logiciel qui fait tourner les LLM **en local** sur ton PC (pas de cloud)	modèles stockés sur `/mnt/data/models_ia/ollama/`
Chunk	Un **morceau** de document (~quelques paragraphes). On découpe car le LLM ne peut pas tout lire d'un coup	**1258 chunks** depuis la réingestion du 04/10/2026 (838 chunks, 106 documents, à l'ingestion initiale du 05/07)
Embedding	La **traduction d'un texte en nombres** : une liste de 384 nombres qui capture le *sens* du texte	modèle `all-MiniLM-L6-v2`
Vecteur	Cette liste de nombres, vue comme une **flèche dans un espace à 384 dimensions**. Deux textes de sens proche → flèches qui pointent presque dans la même direction	1 chunk = 1 vecteur de 384 nombres
Similarité	La mesure « ces deux flèches pointent-elles dans la même direction ? » (angle entre vecteurs). C'est ça, la **recherche sémantique** : par le SENS, pas par les mots exacts	« seuil Newton » retrouve « SEUIL_1NEWTON=20000 » même sans mot commun
Base vectorielle	La base de données qui stocke tous les vecteurs et sait trouver les plus proches **très vite**	**ChromaDB** dans `/mnt/vault_rag/chromadb/`
Collection	Un « rayon » de la base vectorielle (un ensemble de chunks)	`riemann_lab_corpus` (1258 chunks au 04/10/2026 ; 838 au 05/07)
HNSW	L'astuce qui rend la recherche instantanée : un réseau de « raccourcis » entre vecteurs voisins, comme un GPS qui évite de tester toutes les routes	fichier `link_lists.bin`
LlamaIndex	Le **chef d'orchestre** : lit les documents, découpe en chunks, appelle l'embedding, remplit ChromaDB, puis pilote les requêtes	installé dans `zeta_env` (core 0.14.23)
Ingestion	L'opération « remplir la bibliothèque » : documents → chunks → vecteurs → ChromaDB	log : `agent_logs/ingestion_20260705_140201.log`
Corpus	L'ensemble des documents sources ingérés	wiki + code `src/` + prompts ARCHIVE

Terme	Traduction simple	Dans Riemann_Lab
Requête RAG	Question → embedding de la question → recherche des chunks proches → réponse du LLM avec ces chunks	test SEUIL_1NEWTON ✓

---

3. Le schéma complet — de tes documents à la réponse

Phase A — Ingestion (faite le 05/07/2026, ~5 min)



Phase B — Requête (chaque question posée au RAG)

```
Question : « Quel est le seuil SEUIL_1NEWTON et pourquoi ? »
     │
     ▼
 Embedding de la question ⟶ vecteur de 384 nombres
     │
     ▼
 ChromaDB (HNSW) : « quels sont les 3-5 chunks dont le vecteur
   est le plus PROCHE de celui de la question ? »          ~quelques ms
     │
     ▼
 Chunks retrouvés : extraits de analyse_problemes_v13_v15.md,
   Formules_zeta.md …§30
     │
     ▼
 Ollama/mathstral reçoit : [question] + [les chunks]
   « Réponds en t'appuyant sur ces extraits »              ~1-3 min sur CPU i7
     │
     ▼
 Réponse : « SEUIL_1NEWTON = 20 000 car le biais Z_rs ≈ 0.305·t^(-5/4)
   donne une erreur après 1 Newton ≈ biais² ≈ 4e-13 < tol = 1e…-12 »
   + sources citées $\checkmark$
```

Ordres de grandeur sur PC1 : la recherche vectorielle ( ) est quasi instantanée ; c'est la génération mathstral ( ) qui prend du temps (modèle 7B sur CPU, 1-3 min/réponse).

---

4. Les fichiers du SSD — rôle de chacun

Ce qu'on voit dans `/mnt/vault_rag/` après l'ingestion :

Fichier / dossier	Rôle	Analogie
`chromadb/chroma.sqlite3`	**Métastore** : le texte des chunks, leurs métadonnées (fichier source, date), la liste des collections	Le **catalogue** de la bibliothèque
`chromadb/<uuid>/data_level0.bin`	Les **vecteurs** eux-mêmes (1258 × 384 nombres depuis le 04/10/2026 ; 838 × 384 avant)	Les fiches de sens de chaque passage
`chromadb/<uuid>/link_lists.bin`	Le graphe **HNSW** : les raccourcis entre vecteurs voisins	Le plan des rayons pour trouver vite
`chromadb/<uuid>/header.bin`, `length.bin`	Paramètres de l'index, tailles des entrées	Étiquettes techniques
`llamaindex_cache/docstore.json`	Les chunks avec leur texte et provenance, côté LlamaIndex	Photocopies des passages
`llamaindex_cache/index_store.json`	Structure de l'index (quel chunk → quel index)	Sommaire
`llamaindex_cache/graph_store.json`, `image__vector_store.json`	Relations entre docs / images — quasi vides (corpus texte)	—
`corpus/`	Prévu pour des copies locales de sources ; **vide** car l'ingestion lit les sources en place (wiki local, `src/`)	Rayon réservé
`agent_logs/ingestion_*.log`	Journal d'ingestion : docs lus, chunks créés, erreurs	Registre des entrées
`lost+found/`	Système ext4 (réservé root) — **à ignorer**	—

> (!) **Un dossier `<uuid>` = une collection.** S'il y en a deux, l'un peut être un résidu du test hello-world (collection supprimée mais dossier restant) — vérifiable avec `rag_monitor.py`.

---

5. Les tests réels — ce qui a été validé

Test 1 — « Hello world » ChromaDB (05/07, étape 2 du plan)

Le test minimal qui prouve que la chaîne fonctionne :

1. Création d'une collection de test dans `/mnt/vault_rag/chromadb/`
2. Insertion d'1 document → embedding automatique (all-MiniLM-L6-v2, téléchargé dans `~/.cache/chroma`)
3. **Requête sémantique** : la question retrouve le document ✓
4. Suppression propre de la collection

Résultat : écriture/lecture/suppression validées **sur le SSD** ; `chroma.sqlite3` (188 Ko) créé.

Test 2 — Ingestion complète (05/07, étape 3)

Indicateur	Valeur
Documents sources	**106** (59 wiki + 44 code + 3 prompts ARCHIVE)
Chunks créés	**838**
Collection	`riemann_lab_corpus`
Exclusions respectées	`Handoff.md`, `*.bak`, logs
Durée	~5 min 32 s
Script réutilisable	`scripts/rag_ingest_corpus.py` (commit 61ca640)

Test 3 — Première requête RAG (05/07, étape 4)

Question : *« Quel est le seuil SEUIL_1NEWTON et pourquoi cette valeur ? »* → Le retrieval retrouve le **bon extrait** dans `analyse_problemes_v13_v15.md` ✓

C'est la preuve que la recherche **sémantique** marche : le système comprend que la question parle du seuil adaptatif de la Phase 2 Illinois, sans correspondance mot-à-mot exigée.

> **Test de contrôle qualité recommandé** : demander « Quelles sont les IP des 4 machines du cluster ? ». Réponse attendue : `.24 / .52 / .22 / .54`. Si `.94` sort → une archive non taguée a fui dans le corpus (voir règle d'ingestion dans [[etat_rag_brain-vault_20260704]]).

---

6. Surveiller le RAG — quoi observer et pourquoi

Quand le RAG « tourne » (ingestion en cours, ou requêtes), voici les **points de mesure** qui disent si tout va bien :

Point observé	Pourquoi c'est important	Signal d'alerte
Montage du SSD (`mountpoint`)	Leçon du 05/07 : SSD non remonté après reboot → écriture silencieuse sur le disque système	× NON MONTÉ → tout arrêter
Espace libre SSD et racine	Une ingestion qui écrit au mauvais endroit remplit `/`	racine qui se remplit pendant une ingestion
Taille de `chromadb/`	Doit **croître** pendant l'ingestion, être stable sinon	taille figée en pleine ingestion = blocage
Nombre de chunks par collection	La jauge de remplissage de la bibliothèque	compte qui n'augmente plus
Collections présentes	Détecter les résidus (hello-world) ou doublons	collections inattendues
Log d'ingestion (dernières lignes)	Erreurs de lecture, fichiers ignorés	`ERROR`, `Traceback`
Latence de requête	Santé de l'index HNSW	$>$ 1 s pour le retrieval seul = anormal
RAM/CPU	8 Go de RAM : mathstral (7B) + embeddings peuvent saturer	swap intensif (voir §RAM upgrade dans [[STACK]])

(!) **Piège découvert le 05/07 (à retenir absolument)** : le modèle d'embedding utilisé pour la **REQUÊTE** doit être **IDENTIQUE** à celui de l'**INGESTION** (`all-MiniLM-L6-v2` ici). Deux modèles différents = deux « langues de nombres » incompatibles : les distances explosent (~1.6 au lieu de <0.8) et les extraits retrouvés n'ont **aucun rapport** avec la question. Symptôme typique : le retrieval « marche » techniquement mais renvoie n'importe quoi. C'est exactement ce que mesure `--test-query` du script ci-dessous.

Le script **`scripts/rag_monitor.py`** regroupe tout ça :

```
# photo instantanée de l'état du vault
python scripts/rag_monitor.py

# surveillance continue pendant une ingestion (rafraîchi toutes les 10 s)
python scripts/rag_monitor.py --watch 10

# + test de latence d'une vraie requête vectorielle
python scripts/rag_monitor.py --test-query
```

Exemple de sortie :

```
═══
 RAG MONITOR — /mnt/vault_rag — 2026-07-05 14:30 ═══
$\checkmark$ SSD monté (/dev/sdb1)  ·  libre : 227.8 G / 230 G──
 Tailles ──────────────────────────────
chromadb           : 12.4 M
llamaindex_cache   :  3.1 M
agent_logs         : 48.0 K──
 Collections ChromaDB ─────────────────
riemann_lab_corpus : 838 chunks──
 Dernier log ──────────────────────────
ingestion_20260705_140201.log (5 dernières lignes)
  ...
  Nodes (chunks) indexés : 838
```

---

7. Le problème du K et du retrieval imparfait

Section ajoutée après diagnostic du 19/07/2026.

C'est quoi K ?

Quand tu poses une question, ChromaDB retourne les **K chunks les plus proches** (K = nombre fixé à l'avance, ex. K=6). Seuls ces K chunks sont donnés à mathstral pour générer la réponse.

```
Question posée
      |
      ▼
ChromaDB calcule la distance entre la question et les 1258 chunks (838 avant le 04/10/2026)
      |
      ▼
Tri par distance croissante :

  Rang 1  distance 1.116  → chunk quelconque  ← donné à mathstral $\checkmark$
  Rang 2  distance 1.134  → chunk quelconque  ← donné à mathstral $\checkmark$
  Rang 3  distance 1.151  → chunk quelconque  ← donné à mathstral $\checkmark$
  Rang 4  distance 1.163  → chunk quelconque  ← donné à mathstral $\checkmark$
  Rang 5  distance 1.171  → chunk quelconque  ← donné à mathstral $\checkmark$
```

```
Rang 6  distance 1.179  → chunk quelconque  ← donné à mathstral $\checkmark$
─────────────────────────────────────────────────────────────────
Rang 7  distance 1.182  → BON chunk $\times$ jamais vu par mathstral !!!
```

Le bon chunk existe dans la base — mais il n'est pas dans le top-K. Mathstral ne peut pas lire ce qu'on ne lui donne pas.

Pourquoi all-MiniLM-L6-v2 discrimine mal ?

Ce modèle d'embedding a été entraîné sur du **texte anglais généraliste**. Il ne comprend pas bien :

- Le français technique (`"seuil adaptatif de la phase Newton"`)
- Le code mélangé au texte (`SEUIL_1NEWTON=20000` dans un `.c`)
- Les notations mathématiques (`Z_rs`, `?(s)`, `T=100k`)

Résultat : les distances de **tout le corpus** se retrouvent entassées entre 1.1 et 1.2 — le modèle ne fait plus la différence entre un chunk pertinent et du bruit.

Solutions possibles

Solution	Difficulté	Effet
Augmenter K (ex. 6→15)	Facile, 1 ligne	Le bon chunk entre dans le top-K, mais plus de bruit pour mathstral
Changer de modèle d'embedding	Moyen	Meilleure discrimination, mais **ré-ingestion complète obligatoire**
Améliorer le chunking (garder code + contexte ensemble)	Long	Chunks plus riches sémantiquement

(!) Changer le modèle d'embedding = tout refaire : les 1258 vecteurs actuels (838 avant le 04/10/2026) deviennent inutilisables car calculés avec l'ancien modèle. Les deux modèles parlent des ”langues de nombres” incompatibles.

---

8. Le problème de l'hallucination — session du 19/07/2026

Ce qui a été découvert lors du premier test génératif complet avec mathstral.

Les 2 questions de validation

Q1 — *”Quelle est la valeur de SEUIL_1NEWTON ?”* Réponse attendue : `20000` Résultat : ✓ correct — par chance (chunk voisin `illinois_arb.c` dans le top-K)

Q2 — *”Quelles sont les IP des 4 machines du cluster ?”* Réponse attendue : `.24 / .52 / .22 / .54` Résultat : × **mathstral invente 2 IP sur 4** (`.53` / `.25` au lieu de `.52` / `.22`)

Pourtant le retrieval de Q2 était **bon** (distances 0.99–1.12, les bons chunks étaient là).

Leçon clé : un bon retrieval ne garantit pas une génération fidèle sur des données factuelles précises. Mathstral ”lit” correctement les chunks mais **invente quand même**.

Tentative échouée : forcer la citation des sources

Idée : ajouter dans le prompt *"cite le fichier source après chaque fait"*.

Résultat : **PIRE** — retest Q2 → **4/4 IP fausses** (contre 2/4 sans consigne), avec citations correctes des fichiers sur chaque ligne. La citation d'une source réelle n'empêche pas l'invention de la valeur. Un modèle 7B local ne peut pas s'auto-vérifier.

```
Exemple de réponse avec consigne de citation :
  PC1 : [IP_inventée]  [Architecture-Cluster-Zeta.md]   ← IP fausse, fichier réel
  PC2 : [IP_inventée]  [Architecture-Cluster-Zeta.md]   ← IP fausse, fichier réel
  PC3 : [IP_inventée]  [Architecture-Cluster-Zeta.md]   ← IP fausse, fichier réel
  PC4 : [IP_inventée]  [Architecture-Cluster-Zeta.md]   ← IP fausse, fichier réel
```

Solution retenue : vérification programmatique

Fonction `valeurs_non_ancrees()` ajoutée dans `scripts/rag_query.py` (commit `01934df`) :

```
# Principe : comparer chaque nombre/IP de la réponse
# au texte BRUT des chunks récupérés
# → si une valeur est dans la réponse mais absente des chunks → signalée

valeurs_non_ancrees(reponse, chunks_recus)
# $\checkmark$ détecte 2/2 IP fausses (sans consigne de citation)
# $\checkmark$ détecte 3/3 IP fausses (avec consigne de citation)
```

Limite connue : ne détecte pas une valeur correcte attribuée à la mauvaise entité (le chiffre `.52` existe dans les chunks, mais ce n'est pas l'IP de PC3).

Bugs de détection corrigés

Deux faux positifs trouvés et corrigés dans la même session :

Bug	Symptôme	Fix
Comparaison chemin complet vs basename	`illinois_arb.c` ≠ `src/.../illinois_arb.c` → fausse alerte	`os.path.basename()` des deux côtés
Séparateurs de milliers	mathstral répond `20,000.0` pour `20000.0` → fausse alerte	`normaliser_separateurs_milliers()`

> Instabilité mathstral documentée : la même valeur numérique peut sortir sous 3 formats différents selon les appels (`20000.0` → `20,000.0` → `20.000`). À garder en tête pour tout test de validation RAG.

---

9. État du chantier RAG — bilan au 19/07/2026

Étape	État	Date
1. Corpus poussé sur inbox-ia + manifeste	✓	05/07
2. Stack installée + hello-world SSD	✓	05/07

Étape	État	Date
3. Ingestion complète (106 docs → 838 chunks)	✓	05/07
4. Première requête RAG retrieval (SEUIL_1NEWTON)	✓	05/07
5. Script `rag_query.py` industrialisé (retrieval + génération)	✓	19/07 (`86ab0ae`)
6. Test génératif complet Q1+Q2 avec mathstral	✓	19/07
7. Garde-fou anti-hallucination `valeurs_non_ancrees()`	✓	19/07 (`01934df`)
8. Fix faux positifs (basename + séparateurs milliers)	✓	19/07 (`3307bad`)
9. Améliorer le retrieval (K, modèle embedding, chunking)		—
10. Parcours Skilljar (API → MCP → Subagents → Agent Skills)		—
11. Réingestion du corpus : **1258 passages** (contre 838) ; `rag_query.py` avec `--k 4` par défaut et avertissement si le prompt dépasse la fenêtre d'Ollama (4096 tokens)	✓	04/10

Chantier RAG 3 blocs considéré clos. La base est industrialisée et durcie. Prochaine décision : améliorer le retrieval (§7) ou passer au parcours Skilljar.

Voir aussi

- [[etat_rag_brainvault_20260704]] — le bilan d'audit et le plan en 5 étapes
- [[Guide-Ollama-Pratique]] — config matérielle, incidents OOM, benchmarks
- [[ORGANISATION_FICHIERS]] — où vit quoi (règle d'ingestion, inbox-ia)
- [[STACK]] — outils, matériel, roadmap Objectif 2
- [[Bonnes-Pratiques-Claude-Code]] — gestion contexte/tokens
- `scripts/rag_query.py` — script RAG industrialisé (commit `3307bad`)

Guide-RAG-BrainVault-Debutant.md · wiki racine · branche master · hprzeta · MAJ 2026-11-11 · 375 lignes